第8章
指针和引用

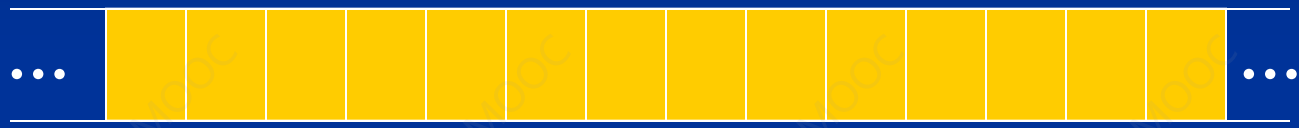
中国大学MOOC

学习目标

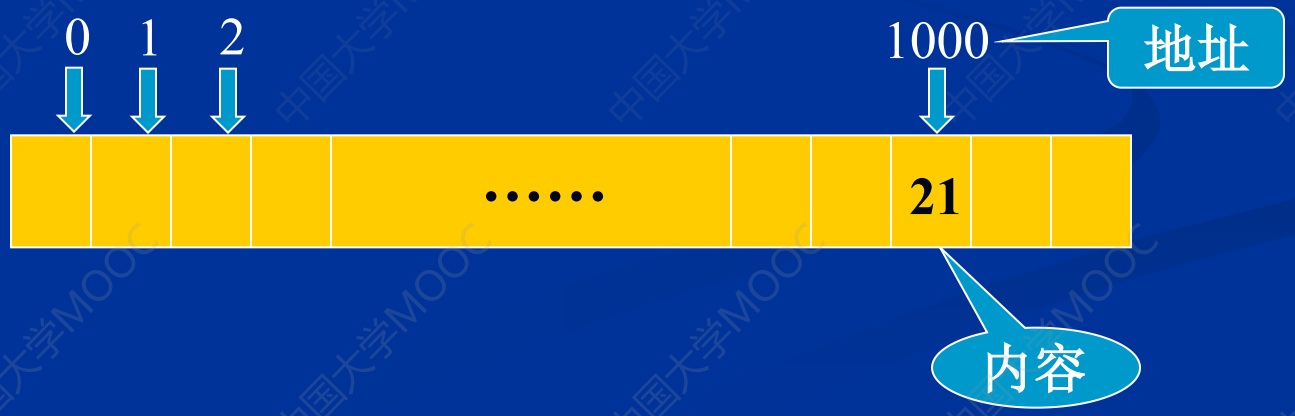
- 掌握指针和引用的的基本概念。包括指针和引用的定义方法与初始化等。
- 搞清楚指针与数组下标的关系。
- 掌握指针和引用作为函数的参数的用法.
- 搞清楚引用传递与值传递的区别。
- 掌握内存的动态申请与释放的方法。
- 掌握链表的基本概念，以及对链表的各种操作。

8.1.1 地址的概念

内存就是内部存储器，是由存储单元组成的。它的特点是存储单元连续。存储单元的最小单位是字节。



与每个住房都有门牌号码一样，为了访问内存中的某个存储单元，则为它们编号，这种编号称为内存地址。通过地址我们就能访问该地址所标识的存储单元。

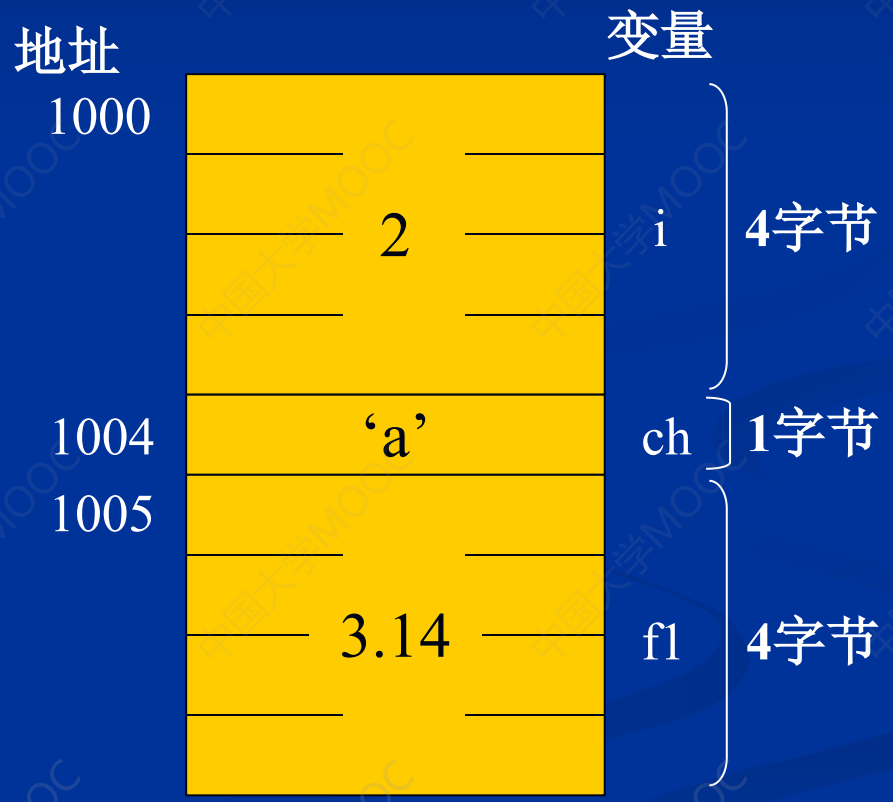


8.1.1 地址的概念

- 在程序中定义一个变量实际就是给了一块内存，该块内存的大小由变量的类型决定。例如在32位的操作系统中，整型占4个字节……变量在内存中总是占连续的几个字节（字符型占1个字节），开始字节的地址就是变量的地址，一旦定义就不再变化。这个地址对用户而言应是透明的。

8.1.1 地址的概念

```
int   i;
char ch;
float f1;
i=2;
ch='a';
f1=3.14;
```



8.1.1 地址的概念

- 整型变量 i , j , k ; 在编译时为变量分配内存

(1) 假设变量 i,j,k 被分配的内存地址分别为2000，2004，2008

(2) 给变量赋值为 i=3; j=6; k=9;

(3) 通常，在程序中通过变量名对变量 进行存取。

•其实程序经过编译后已将变量名转换为变量的地址，对变量值的存取都是通过变量的地址进行的。



8.1.1 地址的概念

访问变量的两种方式：

(1) 直接访问方式

按变量地址存取变量值的方式。

如果读取变量 i的值，直接到为变量i分配的存储单元（2000、2001、2002、2003字节）中取出i的值（3）即可。

(2) 间接访问方式

先将变量 i 的地址存放在另一个变量p中，如果读取变量 i的值，先找到变量p，从p中取出内容（2000，即变量i的起始地址），然后到2000、2001、2002、2003字节中取出i的值（3）。

即通过变量p访问为变量i分配的内存单元。

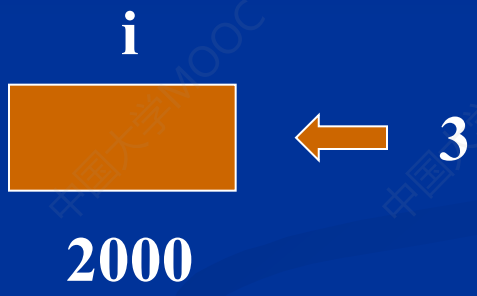
8.1.1 地址的概念

表示将数值3送到变量i中，可有两种表达方法：

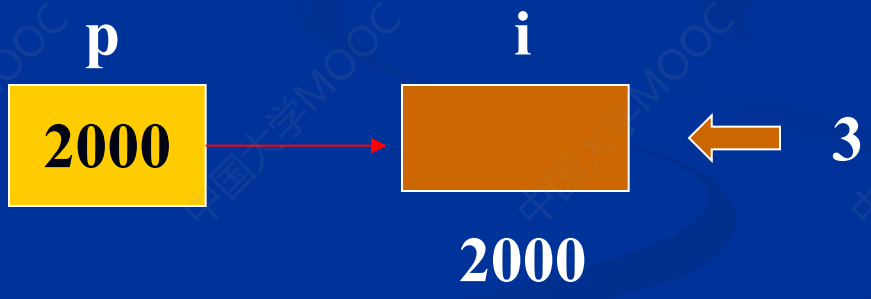
（1）将3送到变量i所标志的单元中。即直接访问方式

（2）将3送到变量p所指向的单元（变量i）中。即间接访问方式

直接访问示意图



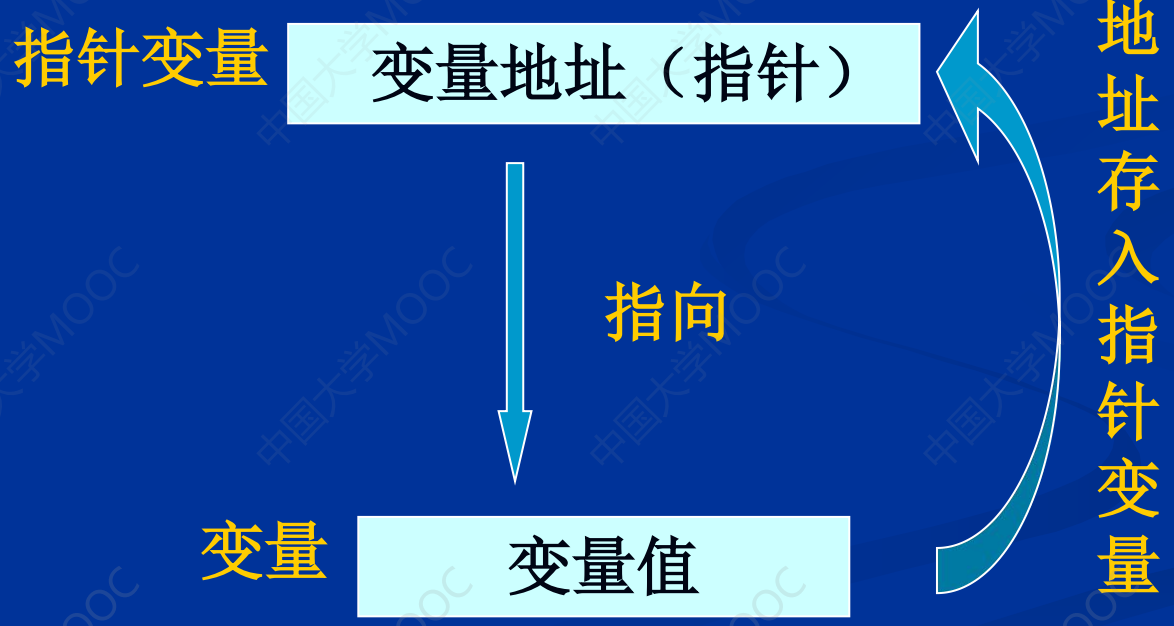
间接访问示意图



注意：我们并不关心变量p的存储地址，对 变量p的访问属于直接访问。

8.1.2 指针变量

- 变量的地址就是指针。若一个变量专门存放内存单元的地址（指针），则该变量为指针变量。



8.1.2 指针变量

- 指针定义格式：

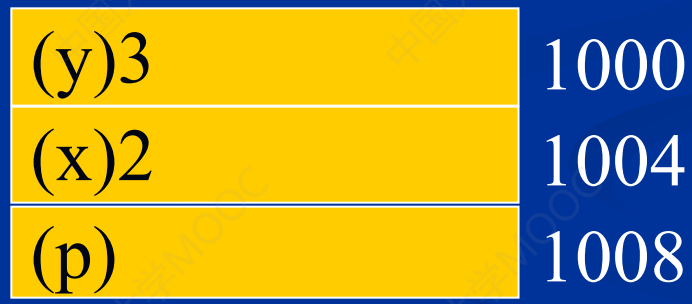
数据类型　*指针变量名1《，*指针变量名2…》；

所指对象所具有的数据类型。指针变量只能存放某一种数据类型的变量，由此可将指针变量划分为整型、实型、字符型指针变量等。

8.1.2 指针变量

- 对一个计算机系统，表示一个地址所需的位数是不变的，比如32位的系统，意味着地址也只要32bit（四个字节），所以所有的指针变量所需的字节数是相同的。

```
int  *p, x, y;
x=2;
y=3;
```



这里是示意性的，在大部分系统，说明的同类型变量是连续的一段空间，不是同类的，可能连续，也可能不连续。与编译系统有关。

8.1.2 指针变量

- 定义指针的注意问题：

  例：int *p1,*p2;

  - 变量名前面的“*”不能省略；
  - 定义中“*”表示所定义的变量是指针变量，但指针变量名是p1，p2，而非*p1，*p2
  - 指针变量只能指向定义时所规定类型的变量。
  - 定义指针变量后，并未确定该变量指向何处，也就是说该变量是不确定的。在引用指针变量前，必须首先让它指向一个变量，这一点非常重要。

8.1.3 指针的运算

- 指针的&和*运算：
  - 变量取地址运算符&的作用是取变量的地址。例如：&i表示求变量i的地址；
  - 递引用（取内容）运算符*的作用是取指针所指向变量的内容。例如：*p表示求指针p所指向变量的内容。
  - &运算和*运算是一对互逆运算。
  - &运算符和*运算符优先级相同，按自右向左方向结合。　　&*p　和　*&i

8.1.3 指针的运算

```
int i;
int *p;
i=3;
p=&i;
*p=15;
```

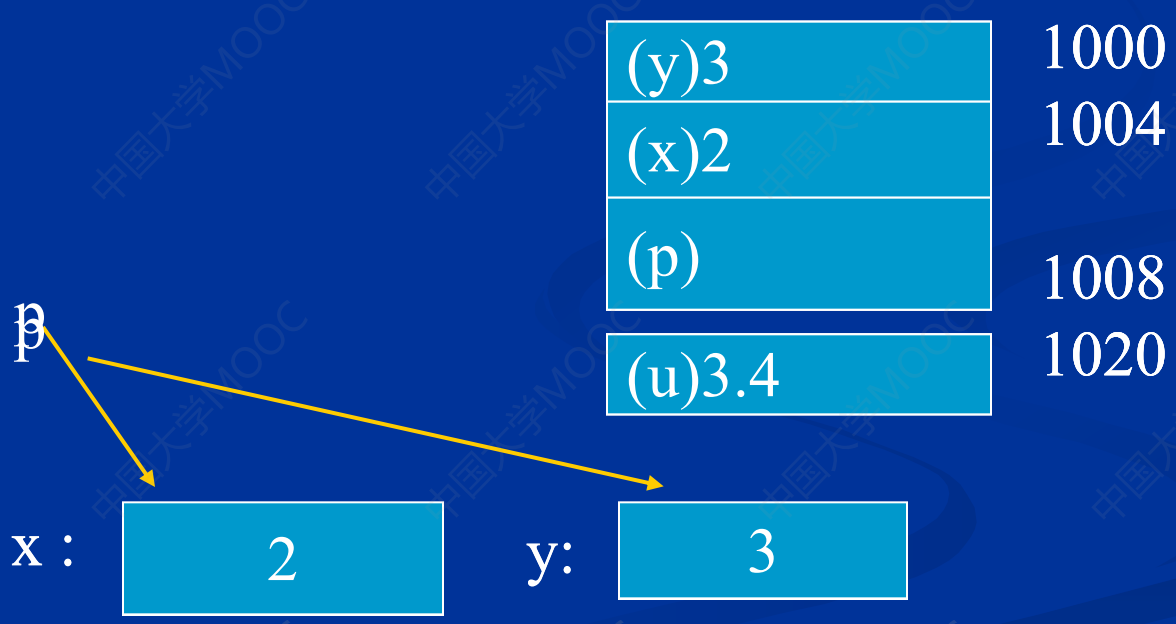


*p=15等价于i=15

8.1.3 指针的运算

- 指针赋值运算：
  a. 使用&运算符把地址值赋值给指针变量

```
int  *p  ,x , y;
double  u=3.4;
x=2;
y=3;
p=&x;
p=&y;
p=&u; //错误
```



指针所指对象：*指针变量
前面例子：　p的值：1000（地址），　*p的值：3

单选题 1分

设置

设有语句 int k=8，*p=&k; *p 的值是

A 指针变量 p 的地址值

B 变量 k 的地址值

C 变量 k 的值 8

D 无意义

提交

中国大学MOOC

单选题 1分

设置

下面哪一个初始化是正确的?

A float f; int *p = &f;

B int *p = 0x3000 ;

C int k, *p = &k;

D int k, &p = &k;

提交

中国大学MOOC

单选题 1分

设置

下面代码段执行输出结果为_____。

```
int a = 2, *pa = &a;
int b = 3, *pb = &b;
*pa*=*pa**pb;
cout<<a<<endl;
```

A 2

B 6

C 12

D 语法错

提交

8.1.3 指针的运算

- 指针赋值运算：
  b. 把指针变量的值赋给另一个指针变量

```
int *p1,*p2,x;
x=2;
p1=&x;
p2=p1;
```

8.1.3 指针的运算

- 指针赋值运算：
  - c. 给指针变量赋值为 NULL（#define NULL 0），NULL是一个空指针，表示该指针变量不指向任一单元。为了避免指针变量的非法引用，一般给没有被初始化的指针变量赋值为NULL。

```
int *p1=NULL;
char *p2=0;
int x;
```

单选题 1分

设置

下面哪一个值不能表示空指针?

A NULL

B 0

C nullptr

D ptrnull

提交

中国大学MOOC

8.1.3 指针的运算

- 指针算术运算：
  a. 指针和整数可进行加或减，若p指向某数组的一个元素，则加或减n代表其后或前n个对象的地址。(包括++，--）

〈指针变量〉=〈指针变量〉+n；



〈指针变量〉=〈指针变量〉+sizeof(〈该指针变量的类型〉)*n；

8.1.3 指针的运算

- 指针算术运算：
  b. 两个指针可做相减运算，若p,q指向同一数组中的元素，则差为它们之间的元素个数。

中国大学MOOC

8.1.3 指针的运算

例如：int a[10] ， *p=a， *q,x;

假定数组a的起始地址为2000,则下列每个操作后变量的值

p=p+1; p的值为2004， p指向数组元素a[1].

q=p; q的值为2004, q指向数组元素a[1].

q+=2; q的值为2012, q指向数组元素[3]

x=q-p; x 的值为2

p++; p的值为2008, p指向数组元素a[2].

++p; p的值为2012 , p指向数组元素a[3].

--p; p的值为2008, p指向数组元素a[2].

p--; p的值为2004, p指向数组元素a[1].

指针的算术运算
除了用于数组外
没有什么意义

得到：
从p到q的数组元素的个数

通过++ 运算，
让指针变量指向数组中
的下一个元素。

通过 --运算，
让指针变量指向数组中
的上一个元素。

8.1.3 指针的运算

- ++(--)注意的问题：

```
int  a[]={3，5，7}, b, *p;
p=a;  // p=&a[0];
```

b=*p++;
(b为3,p为&a[1]）

b=*++p;
(b为5,p为&a[1]）

b=（*p）++;
(b为3,p为&a[0]，a[0] 为4）

数组a



b=*（p++）;
(同 b=*p++;)

b=++*p;
(b为4,p为&a[0]，a[0] 为4）

单选题 1分

设置

执行下面代码段后，b的值为_____。

```
int a[] = {1,2,3,4,5,6,7,8};
int * p = a + 4;
int b = 4+ ++*p++;
```

A 8

B 9

C 10

D 语法错

提交

单选题 1分

设置

执行下面代码段后，*p的值为_____。

```
int a[] = {1,2,3,4,5,6,7,8};
int * p = a + 4;
int b = 4+ ++*p++;
```

A 4

B 5

C 6

D 7

提交

8.1.3 指针的运算

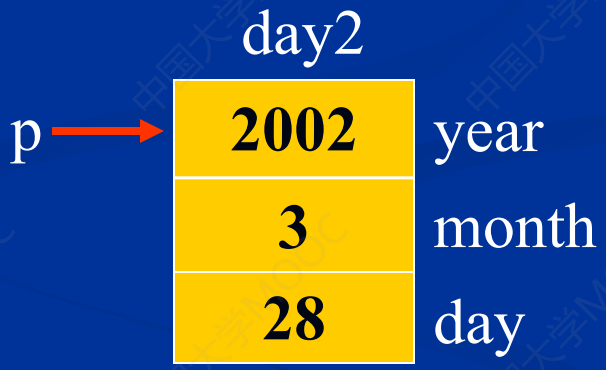
- 指针关系运算：
  a. 在一定条件下，可进行关系比较而言，
     p(op)q
     op:<,>,<=,>=,!=,==,比较的内容为地址
  b. 两个指针值相等则指向相同的内存单元，不等则指向不同的内存单元
  c. 指针变量与0比较时，表示指针变量的值是否为空。

8.2.1 结构的指针

可以定义指针变量来指向已定义的结构变量

```
struct date{
int year;
int month;
int day;
}
date day1={2002,3,28};
date day2;
date *p;
p=&day2;
*p=day1;
```





为了引用p指向的结构的成员，可以用(*p).year的方式。

但C++中提供了一个专门的运算符来处理结构指针，即“->”，于是就有

p->year等价于(*p).year

```
#include <iostream.h>
struct Complex{
        double real,imag;
};
Complex AddComp(Complex *p,Complex *q)
{
        Complex t;
        t.real=p->real+q->real;
        t.imag=p->imag+q->imag;
        return t;
}
void main()
{
        Complex a={1.0,2.0},b={3.0,4.0},x;
        x=AddComp(&a,&b);
        cout<<x.real<<' '<<x.imag<<endl;
}
```

8.2.2 指针作为结构成员

- 指针变量可以作为结构类型的成员，使结构变量能通过指针来关联一个变量。

```
struct Employee{
    …
    Employee *manager;
};
Employee emp1,emp2;
emp1.manager=&emp2;
```

单选题 1分

设置

假设有
struct Person{string name;};
Person p1{“tom”},*p=&p1;
下面哪一个表达式与 p->name 等价?

A p.name

B *p.name

C (*p).name

D *(p.name)

提交

单选题 1分

设置

假设语句：
struct Person { string name; Person *father; };
Person p1{ "tom" }, p2{"jerry", &p1}, *p = &p2;
下面哪一个表达式的值不是“tom”？

A p->father->name

B p1.name

C p2.father->name

D p->name

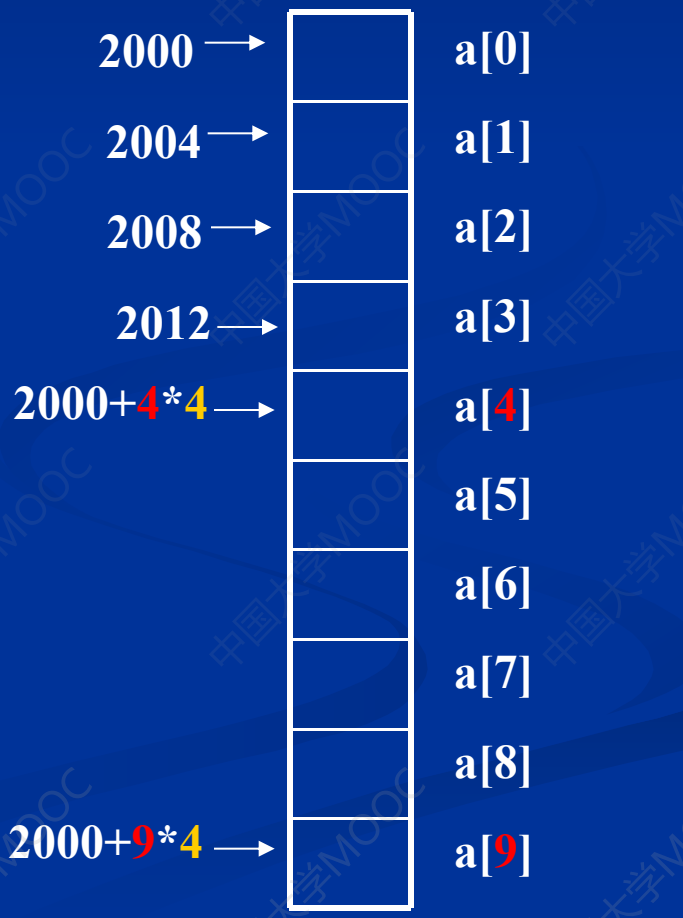
提交

8.3 指针与数组

数组的存储

例如，定义数组 int a[10];

- 数组在内存中占一片连续的存储区，该存储区的大小与数组的元素类型和数组长度有关。
- 每个数组元素占用相同大小的存储空间。
- 利用数组存储区的起始地址，可以逐个获得每个数组元素的存储地址。



注：假设数组a的存储区起始地址为2000。

由图可知，数组元素a[i]的存储地址的计算公式为：
数组a的起始地址+i×元素类型占内存的字节数

8.3.1 用指针访问数组

指向数组元素的指针

例如， int a[10];
int *p;

对指针变量p赋值：p=&a[0]; 则使p指向了数组的第一个元素。

C++规定：

数组名代表数组的起始地址，该地址在程序运行过程中不能修改。 即数组名a是个常量。

因此，下面对指针变量p赋值的两个语句等价：

p=&a[0];
p=a;

因此定义指向数组的指针变量时，可以对其初始化：

int a[10];
int *p=a; (或 int *p=&a[0];)

8.3.1 用指针访问数组

指针和数组的关系

C++中数组与指针关系密切，它们几乎可以互换。数组名可认为是一个常量指针，指针可以完成涉及数组下标的操作。

例如：int a[10] ，*p=a;
则，p指向数组的第一个元素，可用如下的指针表达式引用数组元素a[3]

*(p+3)

式中的 3 是对于指针变量p的偏移量。

当p指向数组的起始位置时，偏移量等于数组的下标。
*(p+3) 这种表示法称为“指针表示法” （或 *(a+3) ）

注意 因为*的优先级高于+，所以圆括号是必需的。

总结

至此，引用一个数组元素，可以采用如下两种方法之一：
(1) 下标法: 如a[i]
(2) 指针法：如 *(p+i) 或 *(a+i)

8.3.1 用指针访问数组

例：输出数组中的全部元素。（采用三种方法输出数组元素的值）

1.下标法

```
#include  <iostream.h>
void main()
{    int      a[10] , i ;
     for(i=0; i<10;i++)
          cin>>a[i];
     for(i=0; i<10;i++)
          cout<<a[i];
 }
```

2. 通过数组名计算数组元素的地址，找出元素的值

```
#include <iostream.h >
void main()
{    int      a[10] , i ;
     for(i=0; i<10;i++)
          cin>>a[i];
     for(i=0; i<10;i++)
          cout<<*(a+i) ;
 }
```

3. 用指针变量指向数组元素

```
#include <iostream.h >
void main()
{    int      a[10] , i , *p;
     for(i=0; i<10;i++)    cin>>a[i];
     for(p=a; p<(a+10); p++)
               cout<< *p;
 }
```

（1）用下标法直观；
（2）用前两种方法效率一样；
编译系统是将a[i]转换为*(a+i)来处理的
（3）用第三种方法省时间，
p++操作快。

编程练习-峰值及其个数

编写程序求峰值个数。有一个数组A，找出A中满足大于或小于左右两个元素的元素，首尾两个元素忽略，例如A={12,45,32,0,14,25,62,48,51}，则峰值有45,0,62,48，峰值个数为4。

要求：编写函数int peak(int *a, int *p, int n)实现在a为首地址的n个大小的数组中发现峰值，并将峰值依次保存在p指向的单元中，返回峰值个数。在以下main函数中实现数组的定义和输入，调用peak函数后将峰值输出。

```
int main(void)
{
   int a[600],p[600],n;
   cin>>n;
   ……   //输入n个整数放入数组a中
   Int m=peak(a,p,n);
   ……   //输出数组p中的峰值
   cout<<m<<endl;
}
```

A 通过测试

B 未通过测试

提交

8.3.1 用指针访问数组

指向多维数组的指针和指针变量

用指针变量也可以指向多维数组，数组名表示的同样是多维数组的首地址。

```
int  a[3][4]; /*假设首地址为2000H  */
```

a		2000H	2004H	2008H	200CH
2000H	a[0]	a[0][0]	a[0][1]	a[0][2]	a[0][3]
2010H	a[1]	a[1][0]	a[1][1]	a[1][2]	a[1][3]
2020H	a[2]	a[2][0]	a[2][1]	a[2][2]	a[2][3]

可以将a数组看作一个一维数组，这个一维数组的每个元素又是一个具有4个int型数据的一维数组，这样，我们就可以利用一维数组的概念来标记一些写法。

8.3.1 用指针访问数组

	a	2000H	2004H	2008H	200CH
2000H	a[0]	a[0][0]	a[0][1]	a[0][2]	a[0][3]
2010H	a[1]	a[1][0]	a[1][1]	a[1][2]	a[1][3]
2020H	a[2]	a[2][0]	a[2][1]	a[2][2]	a[2][3]

a[0]=*(a+0)　a+0为a[0]的地址&a[0]，其值为2000H。

a[1]=*(a+1)　a+1为a[1]的地址&a[1]，其值为2010H。

a[2]=*(a+2)　a+2为a[2]的地址&a[2]，其值为2020H。

8.3.1 用指针访问数组

a[0]为一维数组名，其数组有四个int型的元素：

a[0][0]，a[0][1]，a[0][2]，a[0][3]

同样，它代表一维数组的首地址，所以，a[0]为&a[0][0]。

a[0]为&a[0][0]	a[0][0]=*(a[0]+0)	b[0] = *(b+0)
a[0]+1为&a[0][1]	a[0][1]=*(a[0]+1)	b[1] = *(b+1)
a[0]+2为&a[0][2]	a[0][2]=*(a[0]+2)	b[2] = *(b+2)
a[0]+3为&a[0][3]	a[0][3]=*(a[0]+3)	b[3] = *(b+3)
a[1]+2为&a[1][2]	a[1][2]=*(a[1]+2)	

行　列

8.3.1 用指针访问数组

a		2000H	2004H	2008H	200CH
2000H	a[0]	a[0][0]	a[0][1]	a[0][2]	a[0][3]
2010H	a[1]	a[1][0]	a[1][1]	a[1][2]	a[1][3]
2020H	a[2]	a[2][0]	a[2][1]	a[2][2]	a[2][3]

a为二维数组名，a+1为a[1]的地址，也就是数组第一行的地址，所以a为行指针。

a[1]为一维数组名，a[1]+1为a[1][1]的地址，也就是数组第一行第一列的地址，所以a[1]为列指针。

可以看到：a，a+0，*(a+0), a[0], &a[0][0]表示的都是2000H，即二维数组的首地址。

实际上，a[0], a[1], a[2]并不是实际的元素，它们在内存并不占具体的存储单元，只是为了我们表示方便起见而设计出的一种表示方式。

a为行指针，加1移动一行。

*a或a[0]为列指针，加1移动一列。

a[1]=*(a+1)　　*(a+1)+2=&a[1][2]　　*(*(a+1)+2)=a[1][2]

**(a+1)=*(a[1])=*(*(a+1)+0)=a[1][0]

(a+1)[1]=(*(a+1)+1)=a[1][1]

*((a+1)[1])=**(a+1+1)=**(a+2)=a[2][0]

注意二维数组的各种表示法，a为常量。

数组地址和元素的表示法

表示形式	含义
a	二维数组名，表示数组的起始地址，数组第0行的地址
a+0	第0行的起始地址
a[0]	第0行第0列元素的起始地址
*a, *(a+0)	第0行第0列元素的地址
**a, **(a+0), *a[0], *(*(a+0)+0)	表示元素a[0][0]
a+i, &a[i]	第i行的起始地址
a+i+j, &a[i]+j	第i+j行的起始地址
a[i], *(a+i)	第i行第0列元素的起始地址
*(a+i)+j, a[i]+j, &a[i][j], * &a[i]+j	第i行第j列元素的地址
((a+i)+j), *(a[i]+j), *(&a[i][j]), *(* &a[i]+j), a[i][j]	第i行第j列元素的值

单选题 1分

设置

下面哪一个表达式不能访问二维数组 b 的第 i 行第 j 列元素?

A b[i][j]

B *(b[i]+j)

C *(*b+i)+j

D (*(b+i))[j]

提交

中国大学MOOC

8.3.1 用指针访问数组

指向一维数组的指针变量

```
int  a[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};

int (*p)[4];

p=a;
```

其中(*p)指明p是一个指针变量，再与[4]结合，表示该指针变量所指向的数据是一个一维数组，该数组由四个元素组成。

((p+i)+j)　等同于　*(p[i]+j)　等同于
((a+i)+j)　等同于　*(a[i]+j)　等同于　a[i][j]

8.3.1 用指针访问数组

```
using namespace std;
#include<iostream>
#include<typeinfo>
int main(){
    int b[3][4] = {{11,12,13,14},{21,22,23,24},{31,32,33,34}};
    int (*pb)[4] = b;
    int i;
    cout<<typeid(pb).name()<<endl;
    for(i = 0; i < 3; i++){
        cout<<pb[i][0]<<" ";
        cout<<*(pb[i] + 1)<<" ";
        cout<<*(*(pb + i) + 2)<<" ";
        cout<<(*(pb + i))[3]<<endl;}
    return 0;
}
```

单选题 1分

设置

假设有语句如下：
void f3(int (*p)[4]);
int a[4] = {1,2,3,4};
int b[3][4] = { {1,2,3,4}, {5,6,7,8}, {9,10,11,12}};
下面哪一个调用是非法的?

A f3(&a);

B f3(b[1]);

C f3(&b[1]);

D f3(b);

提交

中国大学MOOC

8.3.2 指针与字符串

在C++中，用两种方法访问字符串：
（1）用字符数组存放一个字符串。
（2）用字符指针指向一个字符串。

（1）用字符数组存放一个字符串。

```
int main()
{    char  string[]="china!" ;
     cout<<string;

 }
```

用字符串常量
初始化字符数组

（2）用字符指针指向一个字符串。

```
int main()
{    char  *string="china!" ;
     cout<<string;

 }
```

用字符串常量
初始化字符指针，
不建议使用

8.3.2 指针与字符串

- 字符型指针变量和字符数组的区别：
  - 内存占用不同；
  - 内容的可变性不同：可直接将字符串输入到字符数组中，而在给字符指针变量赋初值前，不允许将输入的字符串送到指针变量所指向的内存区域；
  - 名字的可变性不同：字符数组起始地址是常量，字符指针值可改变。

8.3.2 指针与字符串

例 将字符串a复制为字符串b

```
using namespace std;
#include <iostream>
int main()
{
        char a[]="I love china!" ,b[20];
        int i;
        for(i=0;*(a+i)!='\0'; i++)
            *(b+i)=*(a+i);
        *(b+i)='\0';
        cout<<b<<endl;
        return 0;
}
```

8.3.2 指针与字符串

```
#include <iostream>
#include <cstring>
using namespace std;
int main()
{
    char s[20];
    char *p="abcdef",*q;
    q=s;
    strcpy(s,"xyz");
    cout<<s<<endl;
    cout<<p<<endl;
    cout<<q<<endl;
    cout<<*p<<endl;
    cout<<*s<<endl;
    cout<<*q<<endl;
    return 0;
}
```

```
xyz
abcdef
xyz
a
x
x
```

投票 最多可选1项

设置

编程练习-字符串中的整数

编写程序输入一个字符串，串内有数字和非数字字符，如：abc2345 345rrf678 jfkld945将其中连续的数字作为一个整数，依次存放到另一个整型数组b中。如对于以上的输入，将2345存放到b[0]，345放入b[1]，...。统计出字符串中的整数个数，并输出这些整数。要求在主函数中完成输入和输出，设计一个函数StrToInt(char *str, int * b)，把指向字符串的指针和指向整数的指针作为函数形参。

```
int main(void)
{
   char str[1000];
   int a[500],num,i;
   cin.getline(str,1000);
   num=StrToInt(str,a);
   for(i=0;i<num;i++) cout<<a[i]<<endl;
   cout<<n<<endl;
}
```

A 通过测试

B 未通过测试

提交

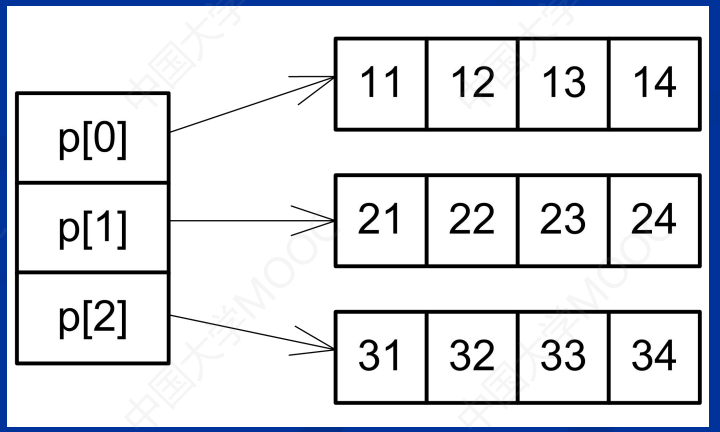
8.3.3 指针的数组

- 指针的数组：
  由若干同类型指针所组成的数组
- 格式：
  <类型> * <数组名>[<数组大小>]
- 如：

```
int b[3][4];
int *p[]={b[0], b[1], b[2]};
char *pcolor[3]={“red”,”yellow”,”green”};
char scolor[3][7]={“red”,”yellow”,”green”};
```

8.3.3 指针的数组

```
using namespace std;
#include<iostream>
int main(){
    int b[3][4] = { {11,12,13,14},{21,22,23,24},{31,32,33,34}};
    int *p[] = {b[0], b[1], b[2]};
    int i;
    for(i = 0; i < 3; i++){
        cout<<p[i][0]<<" ";
        cout<<*(p[i] + 1)<<" ";
        cout<<*(*(p + i) + 2)<<" ";
        cout<<(*(p + i))[3]<<endl;
    }
    return 0;
}
```

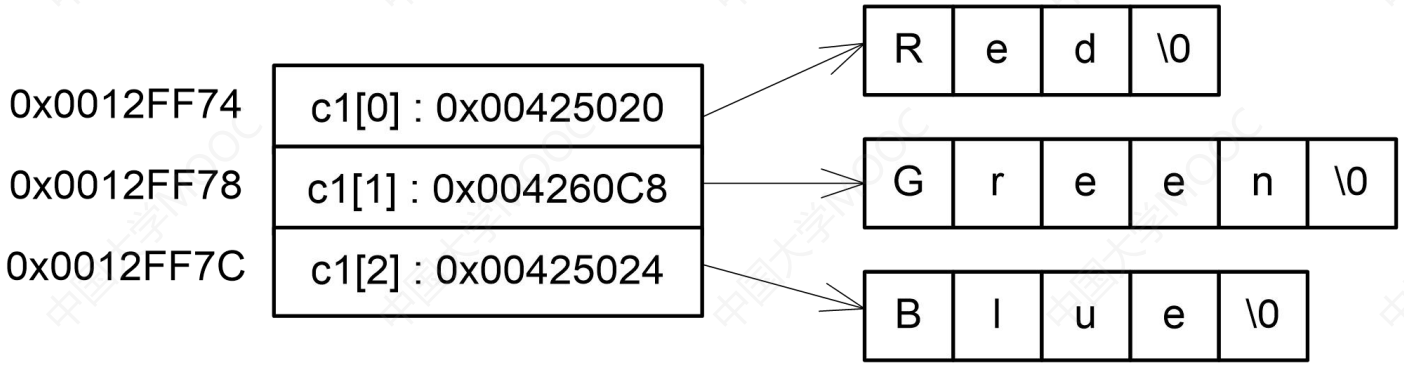


指针的数组用于传递任意行任意列矩阵

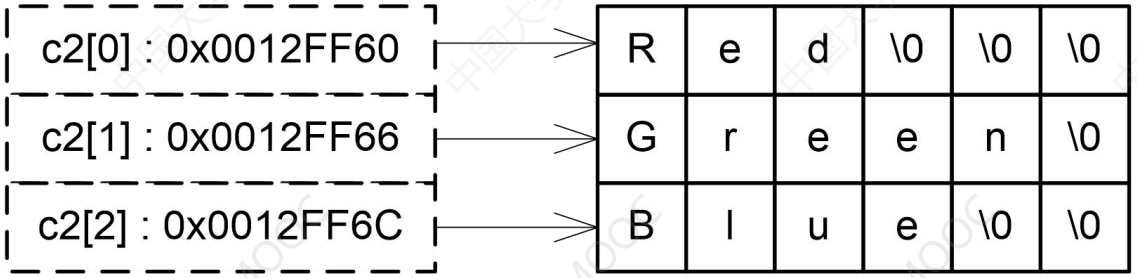
```
void print2D(int *a[], int row, int col){
      for(int i = 0; i < row; i++){
            for(int j = 0; j < col; j++)
                  cout<<a[i][j]<<'\t';
            cout<<endl;
      }
}
int main(void){
      int a[][4]={{1,2,3,4},
                  {3,4,5,6},
                  {5,6,7,8}};
      int *p1[] = {a[0], a[1], a[2]};
      print2D(p1, 3, 4);
      ...
}
```

8.3.3 指针的数组

```
char *c1[] = {"Red","Green","Blue"};
char c2[][6]={"Red","Green","Blue"};
```



(a) 用指针的数组来定义一组字符串



(b) 用二维数组来定义一组字符串

单选题 1分

设置

设有语句如下：
const char *c1[] = {"Red","Green","Blue"};
char c2[][6] = {"Red","Green","Blue"};
32 位系统中下面哪一种说法是正确的?

A sizeof(c1)等于 sizeof(c2)

B sizeof(c1)加 4 等于 sizeof(c2)

C sizeof(c1)大于 sizeof(c2)

D sizeof(c1)加 6 等于 sizeof(c2)

提交

中国大学MOOC

比较一种低效的排序方法，直接交换数组元素

```
void sortString(char str[][20],int n){
        char tmp[20];
        int i,j;
        for (i=0; i<n-1; i++)
         for (j=0; j<n-1-i; j++) {
              if (strcmp(str[j],str[j+1])>0)
                   strcpy(tmp,str[j]);
                   strcpy(str[j],str[j+1]);
                   strcpy(str[j+1],tmp);}
}
```

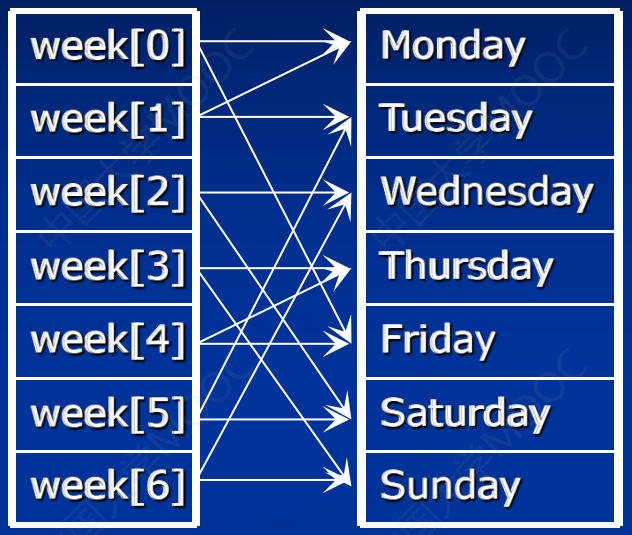
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday

Friday
Monday
Saturday
Sunday
Thursday
Tuesday
Wednesday

交换的是实体数据！代价大

8.3.3 指针的数组

```
using namespace std;
#include <iostream>
#include <cstring>
void SortString(char * str[], int n){
    char *tmp;
    int i, j;
    for (i=0; i<n-1; i++)
        for (j=0; j<n-1-i; j++) {
            if (strcmp(str[j],str[j+1])>0)   {
                tmp = str[j]; str[j] = str[j+1]; str[j+1] = tmp;}
        }
}
int main(void){
    char w[][10] = {"Monday","Tuesday","Wednesday","Thursday","Friday",
    "Saturday","Sunday"};
    char * week[] = {w[0],w[1],w[2],w[3],w[4],w[5],w[6]};
    int num = sizeof(week)/sizeof(char*);
    SortString(week, num);
    for(int i = 0; i < num; i++) cout<<week[i]<<endl;
    return 0;
}
```



比较的是指针所指的内容；
交换的是指针在数组中的位置。

单选题 1分

设置

设有 int b[3][5]; 下面哪一条语句是正确的?

A int (*p)[5] = b;

B int *p[] = b;

C int *p[5] = b;

D (int *)p[5] = b;

提交

中国大学MOOC

单选题 1分

设置

设有语句 int b[3][4]; 下面哪一条语句是正确的？

A int *p[] = {b[0], b[1], b[2]};

B int *p[] = b;

C int *p[2] = {b[0], b[1], b[2]};

D int *p[] = (int *[])b;

提交

中国大学MOOC

8.3.3 指针的数组

- 该指针变量指向另一个指针变量，简称二级指针
- 格式：

  <类型> ** <指针变量名>
- 三级指针变量则加三个“*”

```
int **p,*q,a;
q=&a;
p=&q;
```

8.3.3 指针的数组

```
using namespace std;
#include <iostream>
int main(){
    int b[][4] = { {11,12,13,14},{21,22,23,24},{31,32,33,34}};
    int *p[] = {b[0], b[1], b[2]};
    int **pp = p;
    for(int i = 0; i < 3; i++){
            cout<<pp[i][0]<<" ";
            cout<<*(pp[i] + 1)<<" ";
            cout<<*(*(pp + i) + 2)<<" ";
            cout<<(*(pp + i))[3]<<endl;
    }
     return 0;
}
```

单选题 1分

设置

设有下面语句：
void f4(int **p);
int a[4] = {1,2,3,4};
int b[3][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};
int *q[3] = {b[0],b[1],b[2]};
下面哪个调用是合法的?

A f4(a);

B f4(&a);

C f4(b);

D f4(q);

提交

8.4 指针与函数

（1） 指针做为函数参数

数组或指针作为函数参数的情况：

- 函数的形参为数组，调用函数时的实参为数组名；
- 形参为指针变量，实参为数组名；
- 形参数组名，实参为指针；
- 形和实参均为指针型变量。

```
using namespace std;
#include <iostream>
void swap(int *p ,int *q)
{
   int tmp;
   tmp=*p;
   *p=*q;
   *q=tmp;
}
int main(){
{
   int a=3,b=4;
   swap(&a,&b);
   cout<<a<<' '<<b<<endl;
   return 0;
}
```

8.4 指针与函数

用数组名作函数的参数

```
int main()
 {    int     array[10] ;
      ...
      f(array,10);
      ...
 }
```

```
void  f( int  arr[], int  n)
{
      ...
      ...
 }
```

规定：如果形参数组中元素的值发生变化，实参数组元素的值随之变化。

（1） 实参数组名代表该数组首元素的地址。而形参是用来接收 从实参传递过来的数组首元素的地址。因此，形参应该是一个指针变量（只有指针变量才能存放地址）。

（2） 虽然函数 f 的原型为 void f(int arr[], int n)，但编译时是将arr数组按指针变量来处理的。即：

void f(int arr[], int n) 与 void f(int *arr, int n) 等价

作为函数形参，数组形式和指针形式表示相同的概念。数组和指针交替作为形参和实参，有以下3个等价关系：

- 一级指针等价于一维数组

```
int * → int [n]
char * → char [n]
```

- 数组的指针等价于二维数组

```
int(*)[n] → int [][n]
char(*)[n] → char [][n]
```

- 二级指针等价于指针的数组

```
int ** → int *[n]
char ** → char *[n]
```

注意：二级指针与二维数组不等价

8.4 指针与函数

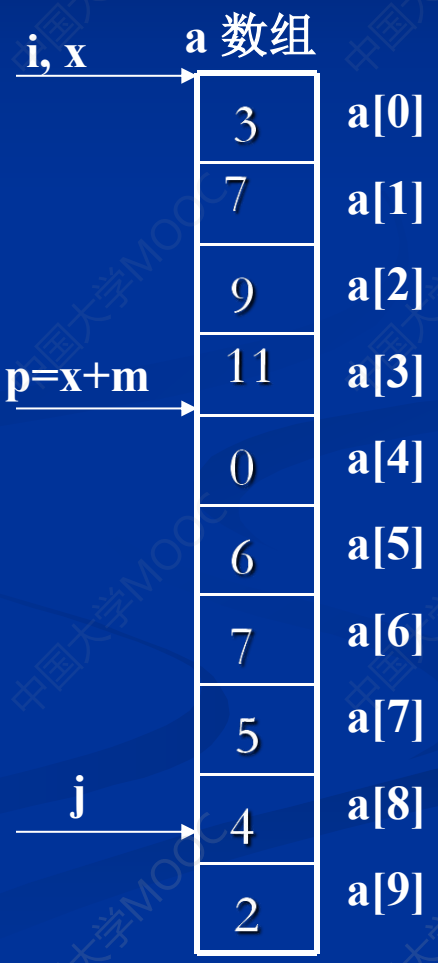
例：将数组a中n个元素按相反的顺序存放。

```
using namespace std;
#include <iostream>

void   inv( int *x, int  n)  /* 形参为指针变量*/
 {     int *p, temp,*i,*j, m=n-1/2;
       i=x; j=x+n-1; p=x+m;
       for(;i<=p; i++,j--)
           { temp=*i; *i=*j;  *j=temp;}
       return;
 }

int main()
 {     int  i,a[10]={3,7,9,11,0,6,7,5,4,2};
       for(i=0;i<10;i++)    cou<<a[i];
       inv(a, 10 );     /* 函数调用，实参为数组名*/
       for(i=0;i<10;i++)    cout<<a[i];

  }
```



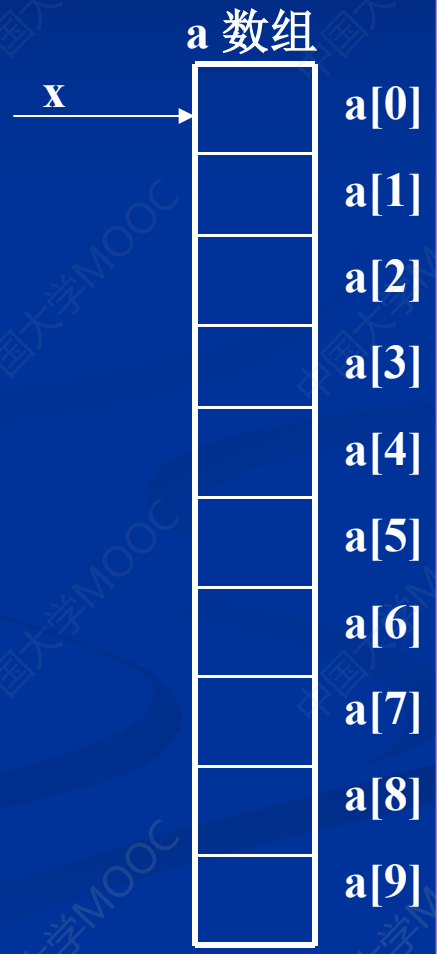
8.4 指针与函数

例:用选择法对10个整数按由小到大顺序排序。

```
using namespace std;
#include <iostream>
  void  sort( int  *x, int  n)  /* 形参为指针变量*/
 {    int  i,j,k,t;
      for(i=0;i<n-1; i++)
        {  k=i;
           for(j=i+1;j<n;j++)
                if(*(x+j)<*(x+k))

                {t=*(x+j);*(x+j)=*(x+k);*(x+k)=t;}
        }
 }
int main()
 {    int  *p,i,a[10];
      p=a;
      for(i=0;i<10;i++,p++)    cin>>*p;
      sort(p , 10 );    /* 函数调用，实参为指针变量*/
      for(i=0, p=a;i<10;i++,p++)   cout<<*p;

 }
```



8.4 指针与函数

例：用函数调用实现字符串a复制。

```
using namespace std;
#include <iostream>
void  copy_string(char  *from, char  *to)
 {
    for(; *from!='\0'; from++,to++)
          *to=*from;
    *to='\0';
 }

int main()
  {    char  a[]="I  love  china!" ,b[20];
       int   i;
       copy_string(a,b);
       cout<<b;

   }
```

此循环可改为：
while((*to=*from)!='\0')
{to++; from++;}

（2）函数返回指针问题：函数可以返回一个指针

```
using namespace std;
#include <iostream>
#include <cstring>
void strcat(char t[],char s[])
{
    int i,j;
    i=j=0;
    while(t[i++]);
    i--;
    while(t[i++]=s[j++]);
}
int main()
{
   char s[20];
   strcpy(s,"abcdef");
   strcat(s,"xyz");
   cout<<s<<endl;
   return 0;
}
```

```
using namespace std;
#include <iostream>
#include <cstring>
char *strcat(char *t,char *s)
{
    char *p=t;
    while(*p++);
    p--;
    while(*p++=*s++);
    return t;
}
int main()
{
  char s[20];
  strcpy(s,"abcdef");
  cout<<strcat(s,"xyz")<<endl;
  return 0;
}
```

投票 最多可选1项 设置

编程练习-查找指定字符

编程实现在字符串中查找指定字符。
要求：编写函数char *findChar(char *s, char c)，利用指针访问方式，在s指向的字符串中查找字符c，若找到，则把字符串中首次出现的c的地址返回，否则返回NULL。在main函数中验证该函数的正确性，输出字符串中c之后的内容，或者输出No found。

```
int main(void)
{
   char s[100],c,*p;
   cin.getline(s,99);
   cin.get(c);
   p= findChar(s,c)
   if(p==NULL)
     cout<<"No found"<<endl;
   else
     cout<<p;
}
```

测试用例：
输入1：
abcde
d
输出1：
de
输入2：
It is a black box
b
输出2：
black box

A 通过测试

B 未通过测试

提交

（3）指向函数的指针：

一个指针若存放的是一个函数的入口地址，该指针称为：**指向函数的指针**

- 定义函数格式： 函数返回类型 （*指针名）（参数表）

```
int (*p)(int,int);
```

- 指针与函数相联系，函数名表示该函数的入口地址：
指针名=函数名；

```
int f(int i,int j) {}
p=f;
```

- 指向函数的指针变量通常只进行赋值操作和关系运算；

- 调用函数格式：
（*指针名）（实参表） 或 指针名（实参表）

单选题 1分

设置

设有说明语句：int * (*fun)(int); fun 表示：

A 一个返回值为指针型的函数名

B 一个指向函数的指针变量

C 一个指向一维数组的指针

D 一个用于指向 int 型的指针变量

提交

中国大学MOOC

单选题 1分

设置

设有语句：
int k=2, *intp = &k;
那么表达式：(*fun)(*intp)是：

A 说明一个函数指针 fun。

B 将 int 变量 k 转换为指针类型 fun。

C 通过函数指针 fun 来调用函数，实参为 *intp。

D 错误表达式

提交

中国大学MOOC

例：求一个函数的累加和,即f(1)+f(2)+...f(n)
其中：f=sin(x) 或 f=1/x

```
using namespace std;
#include <iostream>
#include <cmath>
double f1(double x)
 {
   return 1.0/x;
 }

double sum_f(double (*fun)(double),int n)
{
   double sum=0.0;
   for(int i=1;i<=n;i++)
   sum+=fun((double)i);
   return sum;
}
```

```
int main()
{
   double r1,r2;
    r1=sum_f(f1,10);
    r2=sum_f(sin,10);
    cout<<r1<<' '<<r2<<endl;
}
```

执行下面程序，从键盘输入3<CR>时，程序的输出为 (1) 9
重新执行程序，输入10<CR>时， 则输出为 (2) 。 20

```
using namespace std;
#include <iostream>
 int  fun0(int n)
  {
      return n*n;
  }
int fun1(int n)
  {
    return n+n;
  }
```

```
int main()
{
  int i,(*fp)(int);
  cin>>i;
  if(i<10) fp=fun0;
  else fp=fun1;
  cout<<fp(i)<<endl;
}
```

```
#include <iostream>
using namespace std;
struct node{   int x,y;  };
void sort(node *begin,node *end, bool (*comp)(node,node)){
        int i, j;
        node temp;
        for (i=0; i<end-begin-1; i++)
          for (j=0; j<end-begin-i-1; j++)
              if (comp(begin[j],begin[j+1])){
             temp=begin[j];
             begin[j]=begin[j+1];
                  begin[j+1]=temp;
        }
}
bool xasc(node a,node b){
        if(a.x>b.x)return true;
        return false;}
bool xdesc(node a,node b){
        if(a.x<b.x)return true;
        return false;}
bool yasc(node a,node b){
        if(a.y>b.y)return true;
        return false;}
bool ydesc(node a,node b){
        if(a.y<b.y)return true;
        return false;}
```

```
int main (){
        node
a[10]={567,789,456,7896,35,678,
46,9,35,678,46,678,4,67,46,45,678,234,67,83
4};
        sort(a+2,a+8,yasc);
        for(int i=0;i<10;i++)
        cout<<"a["<<i<<"]="<<a[i].x<<
" "<< a[i].y<<endl;
        return 0;
}
```

指向函数的指针数组：

```
using namespace std;
#include <iostream>
int f1(int x)
{
   return x*x;
}

int f2(int x)
{
   return x+x;
}

int f3(int x)
{
    return  -x;
}
```

```
int main()
{
   int i
   int (*f[3])(int);
   f[0]=f1;
   f[1]=f2;
   f[2]=f3;
   //前面的4行可用下面的代替
   //int (*f[3])(int)={f1,f2,f3};
   cin>>i;
   while(i>=0){
   cout<<f[i%3](i)<<endl;
   cin>>i;}
}
```

输入7，输出？ 14　　输入3，输出？ 9

（4）带参数的main()函数：

- 函数的一般原型：

```
int main(int argc,char* argv[],char* eve[]);
int main(int argc,char** argv,char** eve);
int main(int argc,char* argv[]);
```

- 第一个参数为实际命令行所带的参数个数；
- 第二个参数是一个指向字符串的指针数组，它每一个元素依次指向该命令的一个参数；
- 第三个参数每个元素指向当前运行系统的环境变量。

8.4 指针与函数

```
#include <iostream.h>
int main(int argc, char *argv[]){
    cout<<"argc = "<<argc<<endl;
    cout<<"The program name is "<<argv[0]<<endl;
    for (int i = 1; i < argc; i++)
        cout<<"argv["<<i<<"]="<<argv[i]<<" ";
    cout<<endl;
    return 0;
}
```

单选题 1分

设置

下列哪一个 main 函数原型是错误的?

A main(int argv, char *argc[])

B main(int arc, char **arv)

C main(int argc, char *argv)

D main(int a, char *c[])

提交

中国大学MOOC

8.5.1 void指针

- void型指针可以指向任何类型的变量或函数。

```
void *p; int i=1; float f=1.0f;
p=&f;  p=&i;
```

- 在对void型指针进行间接引用或运算前，必须将其强制转换为某一种具体类型的指针。

```
cout<<*(int *)p;
cout<<*(float *)p;
cout<<*p;   //error
```

8.5.2 const指针

说明方式：

- const在指针变量类型前；

  const int * p;　　//指针变量所指数据是常量

  例：void printAStud(const Student *s)

  unsigned strlen(const char * str)

  int sum(const int *p, int n)

- const在指针变量*后；

  int * const p;　　//指针变量是常量

- 一个const在指针变量类型前，一个在指针变量*后。

在定义重载函数时

- f(int *)与 f(const int*)可重载
- f(const int*)与 f(int*const)可重载
- f(int *)与 f(int*const)不能定义重载函数。

函数调用匹 配规则是

- const 指针实参不能匹配非 const 指针形参，只能匹配 const 指针形参。
- 非 const 指针实参优先匹配非 const 指针形参，若后者不存在则匹配 const 指针形参。

函数可返回 const 指针

- const int * f1()，它要求返回值只能赋给一个 const 指针，如 const int * p1 = f1();
- 通过返回的指针 p1 不能改变指针所指向的内容。

基本约束：

受限定常量不能传给无限定变量，而无限定变量可传给受限定常量。

单选题 1分

设置

设有 int a = 3;下面哪一个指针变量说明能使*pa=5;正确编译运行?

A const int *pa = &a;

B int const *pa = &a;

C int * const pa = &a;

D const int * const pa = &a;

提交

单选题 1分

设置

若有下面4个函数，没有重载错误的是：

```
void f(int *p){};                    //函数1
void f(const int *p){};              //函数2
void f(int const *p){};              //函数3
void f(int * const p){};             //函数4
```

A 函数2与4

B 函数2与3

C 函数1与4

D 均没有错误

提交

单选题 1分

设置

下面哪一个函数编译无错误且无警告：

```
char * f1(){char *a = "hello"; return a;}
const char * f2(){return "hello";}
char [] f3(){ char a[] = "hello"; return a;}
const char [] f4()(return "hello";)
```

A f1

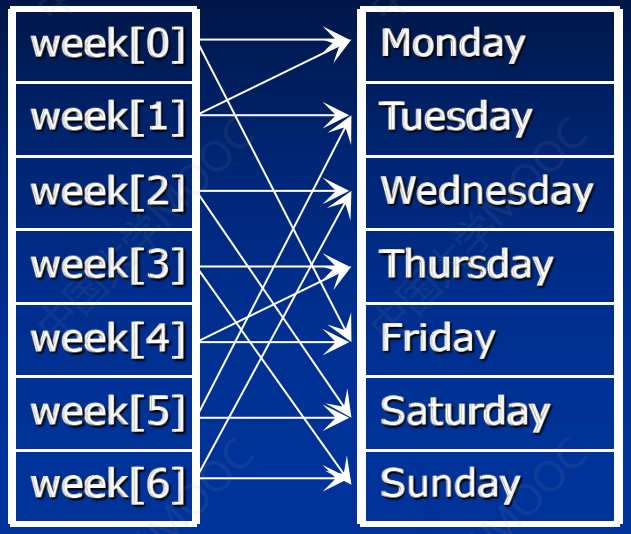
B f2

C f3

D f4

提交

```
using namespace std;
#include <iostream>
#include <cstring>
void SortString(const char * str[], int n){
   const char *tmp;
   int i, j;
   for (i=0; i<n-1; i++)
      for (j=0; j<n-1-i; j++) {
         if (strcmp(str[j],str[j+1])>0)   {
            tmp = str[j]; str[j] = str[j+1]; str[j+1] = tmp;}
      }
}
int main(void){
    const char * week[] =
    {"Monday","Tuesday","Wednesday","Thursday","Friday",
    "Saturday","Sunday"};
    int num = sizeof(week)/sizeof(char*);
    SortString(week, num);
    for(int i = 0; i < num; i++) cout<<week[i]<<endl;
    return 0;
}
```



单选题 1分

设置

设有下面语句：
const char *s1 = "C++ Programming";
char s2[]= "C++ Programming";
下面哪一种操作不会导致错误?

A strcpy(s1, "C++");

B s1 = "C++";

C s2 = s1;

D strcpy(s2, "Java Programming");

提交

中国大学MOOC

指针类型

- 基本类型指针
- 结构体类型指针
- 指向数组的指针
- 指向指针的指针
- 指向函数的指针

中国大学MOOC

指针作为类型

- 结构体中的指针成员
- 指针数组

中国大学MOOC

指针的数组vs数组的指针

指针的数组	数组的指针
一个数组，其中每个元素都是一个指针，这些指针具有相同类型。	一个指针，可指向一种数组，具有确定类型和大小。
<类型名> *[n]	<类型名> (*)[n]
做形参，n可缺省 兼容二级指针：<类型名> * * 不兼容二维数组：<类型名> [m][n]	做形参，n不可缺省 兼容二维数组： <类型名> [m][n]

1.访问二维数组
`int b[3][4];`
`int * p[3]={b[0], b[1], b[2]};`
p与b一样能访问各元素
2.作为函数形参来传递任意行列矩阵
3.处理字符串的数组
4.处理结构的数组，等等
	1.访问二维数组
	`int b[3][4];`
	`int (*p)[4] = b;`
	p与b一样能访问各元素
	2.作为函数形参来传递固定列的矩阵

多选题 1分

设置

```
void f0(char *p){...}
int main(){
 char c = 'c';
 char *pc = "abcd";
 char ca[10] = "abcd";
 ……
}
```

下面哪些调用是非法的？

A `f0(c);`

B `f0(&c);`

C `f0(pc);`

D `f0(&pc[2]);`

E `f0(pc+2);`

F `f0(&ca[2]);`

提交

中国大学MOOC

多选题 1分

设置

```
void f1(int *p){…}
int main(){
  int a[5] = {1,2,3,4,5};
  int b[3][4] =
    {{1,2,3,4},
     {5,6,7,8},
     {9,10,11,12}};
 ……
}
```

下面哪些调用是非法的？

A `f1(a);`

B `f1(&a[2]);`

C `f1(b[1]);`

D `f1(b[1][2]);`

提交

多选题 1分

设置

```
void f2(int *p[]){...}
int main(){
   int a[4] = {1,2,3,4};
   int b[3][4] =
       { {1,2,3,4},
         {5,6,7,8},
         {9,10,11,12}};
   int *q[3] =
     {b[0],b[1],b[2]};
   int *p = a;
   int **pp = &p;
  ……
}
```

下面哪些调用是合法的？

A	f2(a);
B	f2(&a);
C	f2(b[1]);
D	f2(&b[1]);
E	f2(b);
F	f2(q);
G	f2(pp);

提交

多选题 1分

设置

```
void f3(int (*p)[4]){...}
int main(){
      int a[4] = {1,2,3,4};
      int b[3][4] =
      {{1,2,3,4},
       {5,6,7,8},
       {9,10,11,12}};
      int *q[3] =
      {b[0],b[1],b[2]};
 ……
}
```

下面哪些调用是合法的？

A `f3(a);`

B `f3(&a);`

C `f3(b[1]);`

D `f3(&b[1]);`

E `f3(b);`

F `f3(q);`

提交

多选题 1分

设置

```
void f4(int **p){...}
int main(){
      int a[4] = {1,2,3,4};
      int b[3][4] =
      {{1,2,3,4},
       {5,6,7,8},
       {9,10,11,12}};
      int *q[3] =
      {b[0],b[1],b[2]};
 ……
}
```

下面哪些调用是合法的？

A `f4(a);`

B `f4(&a);`

C `f4(b[1]);`

D `f4(&b[1]);`

E `f4(b);`

F `f4(q);`

提交

多选题 1分

设置

```
void f5(int pa[][4]){...}
int main(){
      int a[4] = {1,2,3,4};
      int b[3][4] =
      {{1,2,3,4},
       {5,6,7,8},
       {9,10,11,12}};
      int *q[3] =
      {b[0],b[1],b[2]};
      int *p = a;
      int **pp = &p;
      int (*bp)[4] = b;
  ......
}
```

下面哪些调用是合法的？

A f5(a);

B f5(&a);

C f5(b);

D f5(q);

E f5(bp);

F f5(pp);

提交

8.6 动态使用内存

空间的动态分配与回收：new与delete

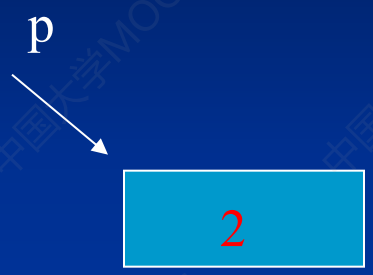
用法：

- 指针变量 = new 类型
- 指针变量 = new 类型（初值）
- 指针变量 = new 类型[表达式]

由系统管理的内存区(称堆)返回一块能够容纳指定类型大小的区域的首地址.不成功返回0（NULL）

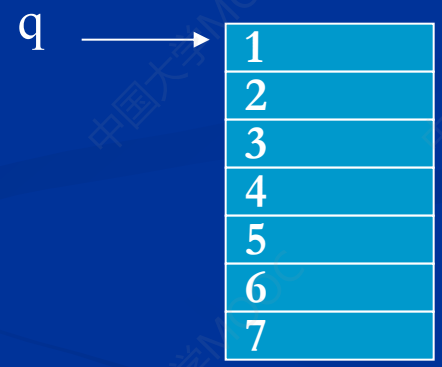
8.6 动态使用内存

```
int *p= new int;
*p=2;
```




```
int *p= new int(2);
int *p= new int{2};
```

同时指定初值2



对数组的分配:

```
int  *q =new  int[7]{1,2,3,4,5,6,7};
```

delete 地址指针 -将先前分配的空间归还系统（空间释放）

delete[ ]地址指针 -释放动态分配的数组空间

8.6 动态使用内存

注意：

- 分配空间后需判断动态分配内存是否成功；

```
int *p= new int;
if(p==0){…}
```

- new计算的指针类型与赋值运算符左操作数类型要一致，否则必须强制类型转换；

```
float (*p)[10];
p=(float(*)[10])new float[10];
```

- 动态分配的空间是不会自动归还系统的，所以要注意不要丢失空间。

```
int *p= new int(2);
p= new  int [6]; 则前面分配的空间丢失。
```

单选题 1分

设置

设有 struct A { int x, y;}; 下面哪一个指针的类型不同于其它三个。

A A *pa1 = new A;

B auto pa2 = new A{ 2,3 };

C auto pa3 = new A[3*4]{{1,2}};

D auto pa4 = new A[3][4]{{{1,2}}};

提交

中国大学MOOC

单选题 1分

设置

设有
int n = 2;
int * a[] = { new int{n}, new int[2*n]{4,5,6}, new int[2*n]{1,2,3} };
a[1][3]的值是_

A 4

B 5

C 6

D 0

提交

8.7 引用

```
using namespace std;
#include <iostream>
void swap(int *p,int *q)
{  int *t;
   t=p;
   p=q;
   q=t;
   cout<<*p<<' '<<*q<<endl;
}
```

```
int main()
{int a=3,b=4;
  int *p1=&a,*q1=&b;
  cout<<*p1<<' '<<*q1<<endl;
  swap(p1,q1);
  cout<<*p1<<' '<<*q1<<endl;
  return 0;
}
```

8.7 引用

```
using namespace std;
#include <iostream>
void swap(int **p,int **q)
{  int *t;
   t=*p;
   *p=*q;
   *q=t;
}
```

```
int main()
{int a=3,b=4;
  int *p1=&a,*q1=&b;
  cout<<*p1<<' '<<*q1<<endl;
  swap(&p1,&q1);
  cout<<*p1<<' '<<*q1<<endl;
  cout<<a<<' '<<b<<endl;
  return 0;
}
```

8.7.1 引用变量的说明及使用

C++中增加了引用来简化上述问题，主要用于函数中以修改实参，内部处理上类同于指针。

定义格式：

类型　&引用变量名 = 变量名;

```
int i;
int &refi=i;
```

注意：

- 定义时必须初始化；
- 初始化值不能为常数；

```
const int &ref=5;
```

- 可用动态分配的内存空间来初始化引用变量；

```
int &ref=* new int;
delete &ref;
```

- 注意各种&和*的不同含义。

单选题 1分

设置

设有 float x=6.2f; 下列哪一条语句能说明 x 的引用?

A float &y = &x;

B float &y = x;

C float y = &x;

D float &y;

提交

单选题 1分

设置

设有说明语句：int m, n=2, &p=n; 下面哪一条语句能完成 m=n 赋值功能?

A m = &p;

B m = &n;

C m = p;

D m = &*p;

提交

8.7.2 引用与数组、指针的关系

1 引用与数组

```
int a[]={1,2,3};
int (&ra)[3]=a;
```

2 引用与指针

```
int i =3;
int *pi = &i;
int *&rpi = pi;
```

不能说明引用的引用，即不存在二级引用

中国大学MOOC

8.7.3 引用与函数

```
using namespace std;
#include <iostream>
void  swap(int &a,int &b)
{
    int t;
    t=a;
    a=b;
    b=t;
}
int main()
{
  int a=3,b=4;
  swap(a,b);
  cout<<a<<' '<<b<<endl;
  return 0;
}
```

原写法：

```
using namespace std;
#include <iostream>
void swap(int *p ,int *q)
{
  int t;
  t=*p;
  *p=*q;
  *q=t;
}
int main()
{
  int a=3,b=4;
  swap(&a,&b);
  cout<<a<<' '<<b<<endl;
  return 0;
}
```

```
using namespace std;
#include <iostream>
void swap(int **p,int **q)
{  int *t;
   t=*p;
  *p=*q;
  *q=t;
}

int main()
{int a=3,b=4;
 int *p1=&a,*q1=&b;
 cout<<*p1<<' '<<*q1<<endl;
 swap(&p1,&q1);
 cout<<*p1<<' '<<*q1<<endl;
 cout<<a<<' ' <<b<<endl;
 return 0;
}
```

利用引用的写法

```
using namespace std;
#include <iostream>
void swap(int *&p,int *&q)
{  int *t;
   t=p;
   p=q;
   q=t;
}

int main()
{int a=3,b=4;
 int *p1=&a,*q1=&b;
 cout<<*p1<<' '<<*q1<<endl;
 swap(p1,q1);
 cout<<*p1<<' '<<*q1<<endl;
 cout<<a<<' '<<b<<endl;
 return 0;
}
```

8.7.3 引用与函数

```
using namespace std;
#include <iostream>
int &f1(void){
   static int count = 1;
   return ++count;
}
int index;
int &f2(void){  return index; }
int main(void){
            f1() = 100;
            for(int i = 0; i < 5; i++) cout<<f1()<<"   ";
            cout << '\n';
            f2() = 100;
            int n = f2();
            cout << "n = " << n << '\n';
            f2() = 200;
            cout << "index = " << index << '\n';
}
```

```
101   102   103   104   105
n = 100
index = 200
```

8.7.4 指针与左值引用的对比

指针*	左值引用&
变量的地址，用&对变量求地址	变量的别名，对变量起别名
指向可变	指向不可变，一次性绑定
存在空指针NULL或nullptr，存在挂空指针问题	不存在空引用，但存在挂空引用问题
允许未初始化的指针	不允许未初始化的引用
有指针的数组*[]	没有引用的数组&[]，引用是单值概念
有指针的指针，即二级指针**	没有引用的引用，&&是右值引用
有指针的引用*&，用指针初始化。	没有引用的指针&*。假设变量a类型为T&，对引用变量a求址&a仍得到T*
假设pa类型为T*， typeid(pa)可识别指针类型T*	假设ra类型为T&， typeid(ra)不能识别引用类型，仍得到T
数组的指针(*)[]，一维数组的指针 不能用范围for遍历元素 用二维数组或一维数组的地址来初始化 可以作为函数返回	数组的引用(&)[]，一维数组的引用 变量自带大小，支持范围for遍历数组元素 用一维数组或二维数组降维来初始化 不可作为函数返回
函数的指针(*)()，可用函数名f或&f初始化	函数的引用(&)()，只能用函数名f初始化 两者功能相同
指针可作为函数形参，用地址作为实参来调用，可用字面量NULL或nullptr来调用。	引用可作为函数形参，用变量作为实参来调用，非常量引用不能用字面量作为实参
函数返回指针，若为非const指针，函数调用解引用后可做左值	函数返回引用，若为非const返回，函数调用可直接做左值

8.7.5 右值引用&&*

- 区别左值和右值
- 赋值运算符左边为左值，右边为右值。++i与--i也是左值。
- 可取地址的、有名字的值是左值。不能取地址的、没有名字的值就是右值。
- 非常量左值引用&不能绑定字面值和表达式，如3， a+3
- 常量左值引用(const&)与右值引用(&&)可绑定字面值和表达式

```
int a = 2;
int && rr1 = a + 3;                    //表达式
auto && rr2 = 4;                       //整数字面值
auto && rr3 = true;                    //逻辑字面值
const auto &&rr4 = 55;                 //const int&&
```

- 前3个引用变量是非常量右值引用
- 最后一个引用变量是常量右值引用

附加 单向链表及其应用

一个线性表A=(1,2,3,4,5)可以用一个一维数组来描述

```
int a[]={1,2,3,4,5};
```

在计算机中，数组a的5个数据是连续存储的，逻辑上的连续性和物理上的连续性是一致的。

缺点：插入或删除一个数据麻烦；数组不能动态的增长。

引入动态链表：表有若干个结点组成，每个结点放一个元素，这些结点在计算机中不一定在一块连续的存储区域中，即物理上可能是不连续的，为了表述逻辑上的连续性，一个结点有存放元素的数据域（数据域可以很复杂，也可很简单，根据应用场合确定）和指针（链）域组成，在链域放逻辑上后一个结点的位置（地址）

```
struct node{
    int data;
    node *next;
};
```

数据域

指针域

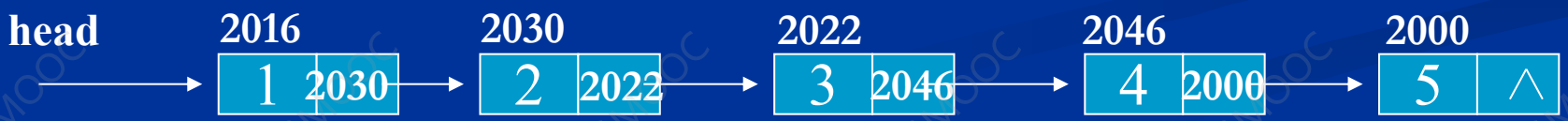
A=(1,2,3,4,5),假设在计算中利用链表存储如右：

2000	5	0
2008		
2016	1	2030
2022	3	2046
2030	2	2022
2038		
2046	4	2000
2054		

假设地址

2016

从示意图，还需一个指针变量能给出第一个结点的地址，该指针变量称：链表头指针（头指针为null时，结点数为0，即空表）



（1）创建一个空链表

```
struct  Node{
            int  data;
            Node *next;
};
struct LinkedList{
            Node * head;
            int size;
};
```

```
//创建一个空链表
LinkedList create(){
            LinkedList list;
            list.head = 0;
            list.size = 0;
            return list;
}
```

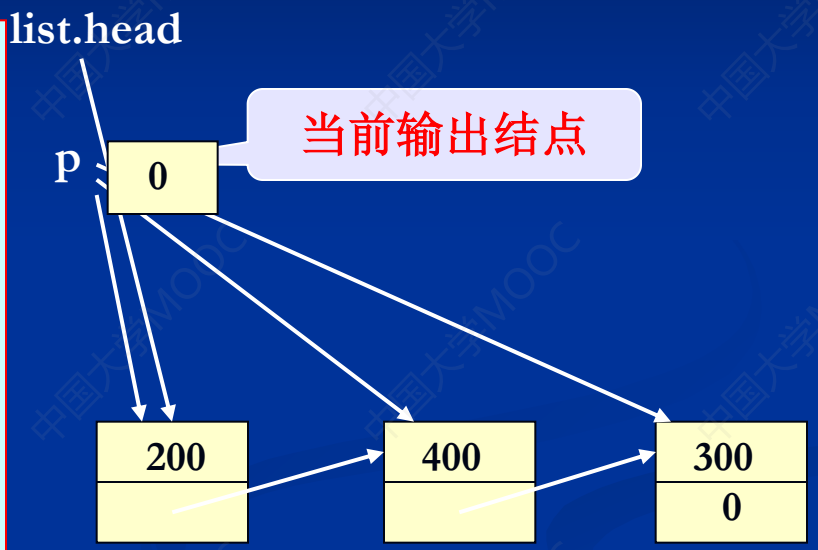
（2）用指定的int数组来创建并初始化一个链表

```
LinkedList create(const int a[], int n)
{
    LinkedList list = create();
    Node *p1, *p2;
    for (int i = 0; i < n ; i++){
            p1 = new Node;      p1->data = a[i];    p1->next = NULL;
            if (list.head == 0)    list.head = p1;
            else    p2->next = p1;
            p2 = p1;
    }
    list.size = n;
    return list;
}
```

假设数组中的值为{200,400,300}

（2）输出链表上各个结点的值

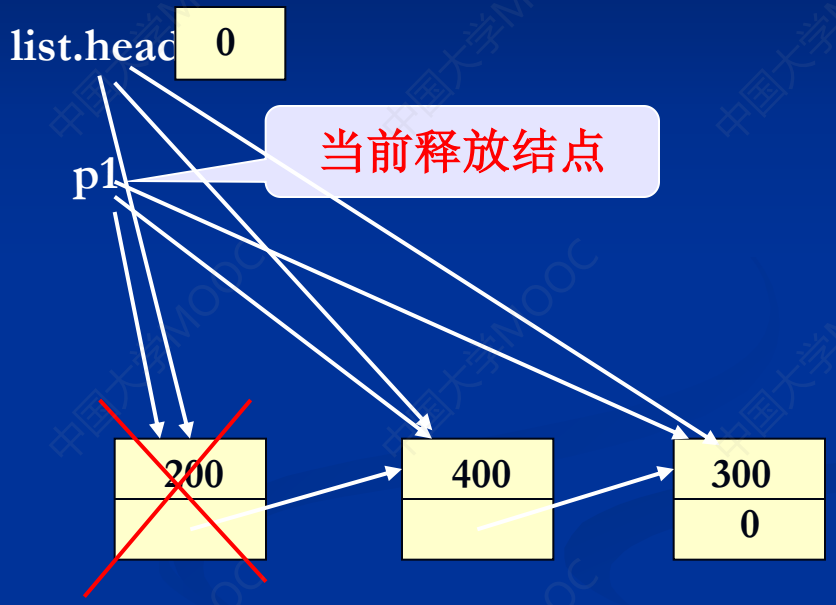
```
void   print(const LinkedList &list)
{
   cout<< "size=" <<list.size<< ":" ;
   Node *p=list.head ;
   while ( p !=  NULL )  {
      cout <<p->data << '\t';
      p= p->next;
   }
   cout <<"\n";
}
```




```
size=3:
 200     400     300
```

（4）释放链表的结点空间

```
void removeAll(LinkedList &list)
{
    node *p1;
    while(list.head){
        p1=list.head;
        list.head=list.head->next;
        delete p1;
    }
    list.size=0;
}
```

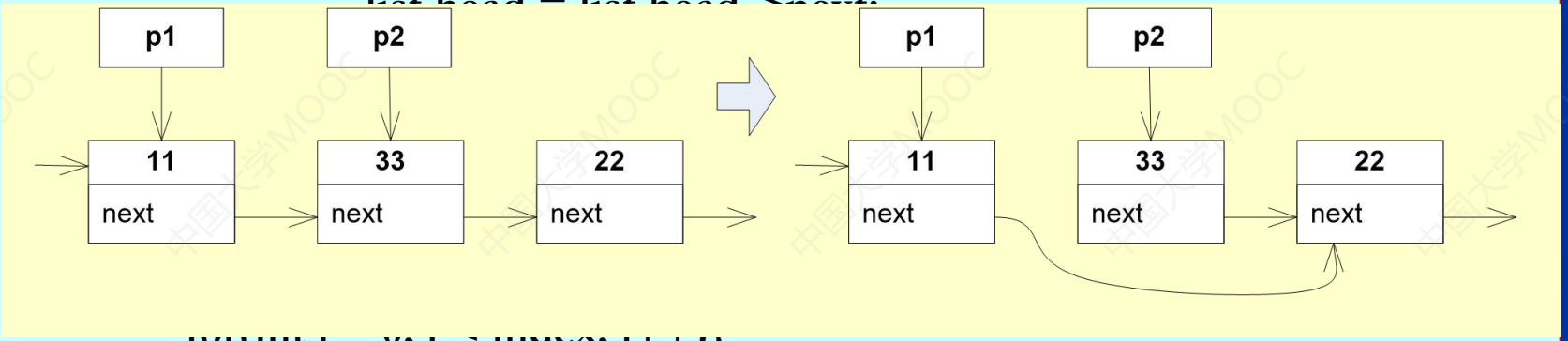


（5）在链表中查找指定的data，返回持有该值的第一个结点的下标位置。如果没有找到，返回-1 。

```
int search(const LinkedList &list, const int data){
        Node *p = list.head;
        for(int i = 0; p != NULL; i++){
                if (p ->data == data)
                        return i;
                p = p -> next;
        }
        return -1;
}
```

（6）删除指定下标index结点，并返回被删结点的数据

```
int remove(LinkedList &list, const int index){
        if (list.size == 0 || index < 0 || index >= list.size)
                throw index;
        if (index == 0){
                list.size--;
                Node *p = list.head;
                list.head = list.head->next;
                delete p;
                return p->data;
        }
        Node *p1 = list.head;
        Node *p2 = p1->next;
        for(int i = 0; i < index; i++){
                p1 = p2;
                p2 = p2->next;
        }
        list.size--;
        p1->next = p2->next;
        int a = p2->data;
        delete p2;
        return a;
}
```

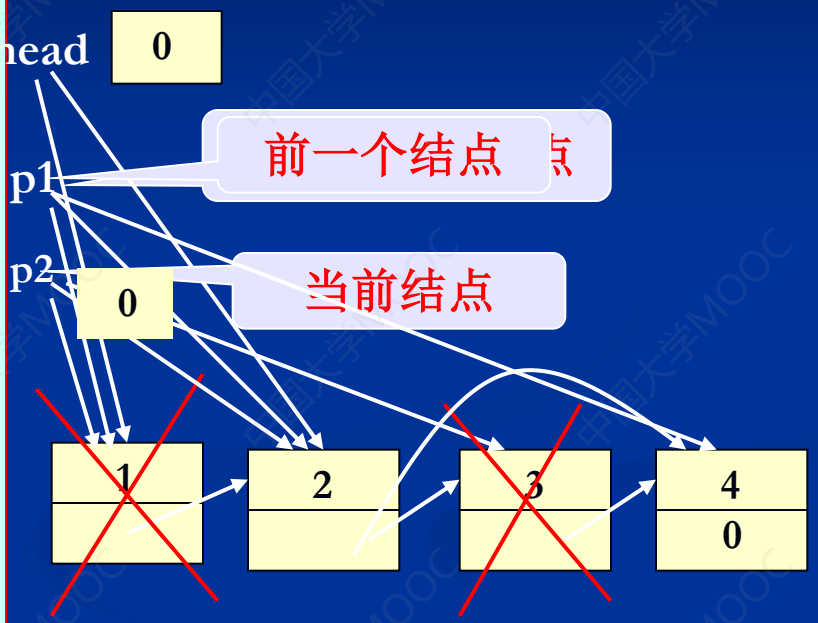


（7）删除链表上具有指定值的一个结点

```
LinkedList Delete_one_node(LinkedList list,int num)
{
   Node *p1,*p2;
   if ( list.size==0 ) {
        cout <<"链表为空，无结点可删!\n";
        return  list;}
   if ( list.head->data == num ){
        p1=list.head;
        list.head = list.head->next; list.size--;
        delete p1;
        cout <<"删除了一个结点!\n";}
   else {
        p2=p1 = list.head;
        while (p2->data != num && p2->next != 0) {
            p1 = p2 ; p2 = p2->next;}
        if ( p2->data  == num ) {
            p1->next = p2->next; list.size--;
            delete p2;
            cout <<"删除了一个结点!\n";}
       else
            cout <<num <<"链表上没有找到要删除的结
点!\n";     }
        return list;
}
```

指定值

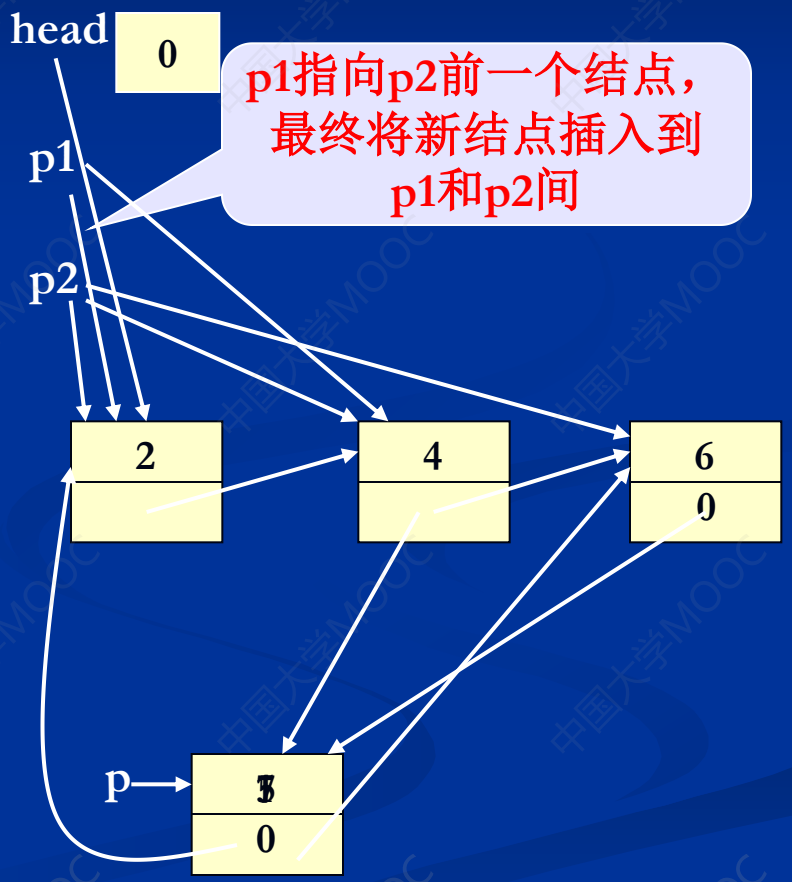
若指定值为3



链表上没有要删除的结点！

（8）结点插入链表，保持升序关系

```
void insertAsc(LinkedList &list, const int data){
    Node *p = new Node;
    p->data = data;
    list.size++;
    if (list.head == NULL || p->data <=
list.head->data){ //作为头结点
            p->next = list.head;
            list.head = p;
            return;
    }
    Node *p1 = list.head, *p2 = p1;
    while (p2 != NULL && p2->data < p->data){
            p1 = p2;
            p2 = p2->next;
    }
    p->next = p2;
    p1->next = p;
}
```



8.8 Lambda表达式*

- Lambda表达式(简称L式)是C11引入的语言构造。以表达式形式表示一个匿名函数，其中包含简短代码，往往作为算法函数调用的实参。

```
double getAverage(int a[], int n) {
  double sum = 0;
  for_each(a, a + n, [&sum](auto x) {sum += x; });
  return sum / n;
}
```

捕获

参数表

函数体

8.8.1语法构造

[捕获]，说明函数体如何从外界捕获变量。[]表示不获取外界变量

捕获外界变量有两种方式：以值捕获=、引用捕获&

- [=] //以值捕获所有外界变量
- [var] //以值捕获var，等价于[=var]
- [&] //引用捕获所有外界变量
- [&total, factor] //引用捕获total，以值捕获factor
- [factor, &total] //与上面一样，次序无所谓
- [&, factor] //用引用访问所有，但factor是以值捕获
- [=, &total] //以值捕获所有，但total是引用捕获
- 如果子句包含&，就不允许再出现“&标识符”；
- 如果子句包含=，就不允许再出现“=标识符”

L式的参数表与函数体

- (参数表)，与函数形参表相同。
- 单个参数(一元)，或两个参数(二元)；
- 以值传递，或者引用传递&(可改变参数)；

- {函数体}，与普通函数体一样，包含一条或多条语句。如果L式做谓词(Predicate)就应返回逻辑值。
- 一般无需说明函数体返回类型。

8.8.2 简单用法

函数体后添加(实参表)，就计算该L式的值：

```
cout<< [](int x, int y) { return x + y; }(5, 4) <<endl;
```

可对L式命名，然后再用“名称(实参)”来调用。例如：

```
int i = 3, j = 5;
function<int(void)> f = [i, &j] { return i + j; };
cout << f() << endl;
```

一个递归L式需要用function类型命名

```
function<int(int)> f1 =
        [&f1](int n) {return n == 1 ? 1 : n * f1(n - 1); };
cout << f1(5) << endl;
```

8.8.3嵌套L式与高阶函数*

高阶函数high order function。

采用另一个L式作为其形参，或其返回表达式中包含另一个L式。

8.8.4调用STL算法

形参名称	所在STL函数	L式要求
Generator	generate	零元无参，返回与元素类型一致的数据
Function	for_each	一元函数，参数x表示遍历到一个元素，无返回
Predicate	find_if count_if	一元谓词，返回逻辑值，参数x表示遍历到一个元素。返回逻辑值，表示是否满足查找条件。
Predicate	sort max_element min_element	二元谓词，参数x1,x2表示前后两个相邻元素。返回逻辑值，表示应满足的排序条件或判断条件。

STL算法适用于一维数组和容器，可处理指针的数组。

中国大学MOOC